Polimorfizm

Polimorfizm

Mechanizm polimorfizmu polega na tym, że tak samo nazwana operacja może oznaczać wykonanie zupełnie innej czynności w zależności od typów argumentów operacji. Np. operator `+` może w wielu językach (Python, Java) oznaczać zarówno dodawanie liczb (całkowite lub zmiennoprzecinkowe) jak i konkatenację (łączenie) napisów. W przypadku języków statycznie typowanych (np. Java) jest to przykład tzw. **polimorfizmu statycznego** — decyzja dotycząca wykonywanej operacji podejmowana jest w trakcie kompilacji.

W kontekście programowania obiektowego dużo częściej używa się terminu polimorfizm do opisania **polimorfizmu dynamicznego**. W tym przypadku decyzja dotycząca tego, która operacja ma zostać wykonana podejmowana jest w czasie wykonania programu. W języku Python (również w Javie) decyzja ta podjęta może być na podstawie typu obiektu, na rzecz którego wykonywana jest metoda. Np. w przypadku wywołania:

```
obiekt.metoda(argument)
```

zostanie wywołana metoda z klasy, do której należy `obiekt`. Język Python jest dynamicznie typowany, więc typ `obiektu` może nie być znany do czasu wywołania metody.

Mechanizm ten dotyczy również operatorów (użycie operatorów w Pythonie oznacza wywołanie metod specjalnych). Np. to, co oznacza instrukcja

```
a + b
```

zależy na pewno od typu obiektu `a` (wywołanie metody `__add__` z klasy obiektu `a`), a być może również od typu obiektu `b` (wywołanie metody `__radd__` z klasy obiektu `b`).

Przykład — klasy definiowane przez użytkownika, metody

Mamy dwie klasy reprezentujące figury geometryczne płaskie. Klasa `Kolo` umożliwia reprezentowanie kół o zadanym promieniu. Klasa `Kwadrat` umożliwia reprezentowanie kwadratów o zadanym boku. Obie klasy mają metody `obwod` i `pole` zwracające odpowiednio obwód figury i jej pole powierzchni.

In [2]:
```
import math


class Kolo:
    def __init__(self, promien):
        self.promien = promien

    def obwod(self):
        return 2 * math.pi * self.promien

    def pole(self):
        return math.pi * self.promien**2
```

In [3]:
```
class Kwadrat:
    def __init__(self, bok):
        self.bok = bok

    def obwod(self):
        return 4 * self.bok

    def pole(self):
        return self.bok**2
```

Definiujemy funkcję `wyswietl_figure` używającą wspomnianych metod.

In [4]:
```
def wyswietl_figure(figura):
    print(f"obwód: {figura.obwod()}")
    print(f"pole: {figura.pole()}")
```

Wywołania metod `obwod` i `pole` są polimorficzne (używają mechanizmu **polimorfizmu**). Nie wiadomo której dokładnie metody dotyczą. Mogą oznaczać wywołanie metod z klasy `Kolo`:

In [5]:
```
wyswietl_figure(Kolo(3.5))
```

```
obwód: 21.991148575128552
pole: 38.48451000647496
```

lub `Kwadrat`:

In [6]:
```
wyswietl_figure(Kwadrat(7))
```

```
obwód: 28
pole: 49
```

Mogą to być również wywołania tych metod z jeszcze innej klasy z metodami `pole` i `obwod`.

W językach statycznie typowanych każdy parametr funkcji ma określony typ. Argument użyty przy wywołaniu musi być odpowiedniego typu. W przypadku funkcji takiej jak figura potrzebowalibyśmy np. typu `Figura`. To, że metoda może przyjmować argumenty różnych typów (dzięki czemu możliwy jest

polimorfizm) jest w takich językach zapewniane przez różne mechanizmy takie jak np. interfejsy (niewystępujące w Pythonie jako konstrukcja językowa) i dziedziczenie (omówione dalej, ale nie jest wymagane do polimorfizmu w Pythonie). W Pythonie to, czy użycie obiektu zadanego typu jako argumentu danej funkcji się powiedzie (zakończy bez błędów) zależy wyłącznie od tego, czy można dla tego obiektu wywołać metody używane przez funkcję. Np. funkcję `wyswietl_figure` można wywołać z dowolnym obiektem mającym metody `pole` i `obwod` (czyli zachowującym się jak figura geometryczna).

In [7]:
```
class A:
    def obwod(self):
        return 5

    def pole(self):
        return 8
```

In [8]:
```
a = A()
```

In [9]:
```
wyswietl_figure(a)
```

```
obwód: 5
pole: 8
```

Liczba `5` nie ma metod `obwod` ani `pole`. Poniższe wywołanie zakończy się wyjątkiem `AttributeError` przy próbie odczytania pierwszej z nich:

In [10]:
```
wyswietl_figure(5)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[10], line 1
----> 1 wyswietl_figure(5)

Cell In[4], line 2, in wyswietl_figure(figura)
      1 def wyswietl_figure(figura):
----> 2     print(f"obwód: {figura.obwod()}")
      3     print(f"pole: {figura.pole()}")

AttributeError: 'int' object has no attribute 'obwod'
```

In [11]:
```
class B:
    pass
```

In [12]:
```
b = B()
```

Podobnie obiekt klasy `B` nie ma wymaganych metod. Wywołanie `wyswietl_figure(b)` również spowoduje `AttributeError`:

In [13]:
```
wyswietl_figure(b)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[13], line 1
----> 1 wyswietl_figure(b)

Cell In[4], line 2, in wyswietl_figure(figura)
      1 def wyswietl_figure(figura):
----> 2     print(f"obwód: {figura.obwod()}")
      3     print(f"pole: {figura.pole()}")

AttributeError: 'B' object has no attribute 'obwod'
```

Zgodność typów określana na podstawie tego czy mają odpowiednie metody nazywana jest Duck typing („kacze typowanie”).

Przykład — typy wbudowane, operatory

Użycie operatora `+` w poniższej funkcji również jest polimorficzne.

In [14]:
```
def suma(a, b):
    return a + b
```

Może on oznaczać dodawanie:

In [15]:
```
suma(4, 5)
```

Out[15]:
```
9
```

Konkatenację:

In [16]:
```
suma("abc", "def")
```

Out[16]:
```
'abcdef'
```

Dowolną inną operację zdefiniowaną jako metoda `__add__` (lub `__radd__`) w odpowiedniej klasie (metody te są dokładniej omówione przy okazji omawiania metod specjalnych):

In [17]:
```
class A:
    def __add__(self, a):
        return A()

    def __repr__(self):
        return "A()"
```

In [18]:
```
suma(A(), A())
```

Out[18]:
```
A()
```